

Eggplant Detection and Counting Across the Entire Greenhouse Using Deep Learning and Motion Parallax

Tatsuya Ishizuka

*Graduate School of Interdisciplinary Information Studies
The University of Tokyo
Tokyo, Japan
tatsuya.ishizuka@koshizuka-lab.org*

Yusuke Sasaki

*Graduate School of Interdisciplinary Information Studies
The University of Tokyo
Tokyo, Japan
yusuke.sasaki@koshizuka-lab.org*

Takeo Hamada

*Graduate School of Interdisciplinary Information Studies
The University of Tokyo
Tokyo, Japan
takeo.hamada@koshizuka-lab.org*

Yuki Shimazaki-Kishi

*Agricultural Research and Development Center
National Federation of Agricultural Cooperative Associations
Tokyo, Japan
shimazaki-yuuki@zennoh.or.jp*

Noboru Koshizuka

*Graduate School of Interdisciplinary Information Studies
The University of Tokyo
Tokyo, Japan
noboru@koshizuka-lab.org*

Abstract—To address labor shortages and population growth, the importance of maximizing productivity through precision agriculture is increasing. As part of precision agriculture, counting the number of fruits on crops is conducted to monitor growth conditions and predict yields. Manual counting is inaccurate and costly, so methods using object detection technologies to count fruits have been proposed. However, existing methods count all visible fruits in the frame, leading to duplicate counts and inaccuracies in measuring the total number of fruits throughout the greenhouse. In this paper, we propose a method for accurately counting the number of eggplants in an agricultural greenhouse by using an object detection model and an object tracking model. Our method utilizes the principle of motion parallax to prevent double counting and ensure accurate counts of each ridge. This approach leads to a more precise estimate of the total number of fruits in the greenhouse. The proposed method was evaluated using object tracking metrics and a comparison with the actual fruit count, and the results showed superior performance compared to existing methods. This demonstrates that our approach is more suitable for estimating the total number of fruits throughout the entire greenhouse.

Index Terms—object detection, multi-object tracking, smart agriculture, yield prediction

I. INTRODUCTION

Improving productivity in agriculture is becoming increasingly important due to labor shortages and population growth. To enhance productivity, precision agriculture, which involves making crop production decisions based on data acquisition and analysis [1], is gaining attention. Precision agriculture often involves balancing the growth of crops between veg-

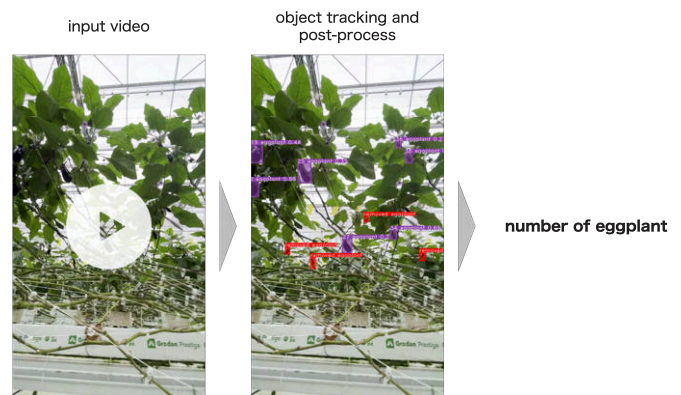


Fig. 1. Our method performs fruit counting using videos of greenhouse ridges and object tracking techniques. Subsequently, as a post-processing step, fruits from ridges not targeted for counting are removed based on the principle of motion parallax. As a result, the method outputs the number of fruits per ridge.

etative and reproductive stages. In fruit vegetables such as tomatoes and eggplants, it is well-known that the crops undergo vegetative growth, which involves the expansion of the plant's own body, and reproductive growth, which involves active fruiting. The tendency of a crop towards vegetative or reproductive growth can be determined by parameters such as the thickness of the stem, the height of flower placement, and the number of fruits [2]. These parameters are crucial because adjusting factors such as temperature, irrigation levels, and

the extent of leaf pruning according to whether a crop is in a vegetative or reproductive growth phase can help achieve a balanced growth and increase yields. Among these parameters, understanding the number of fruits on the crops is particularly important. Knowing the number of fruits not only allows us to determine whether a crop is leaning towards vegetative growth or reproductive growth, but it also enables us to assess the yield that can be shipped soon, thereby aiding in the planning of sales strategies.

Manual measurement of fruit counts not only incurs high labor costs but is also inaccurate. In modern, large-scale greenhouses, which often span over 100 meters by 100 meters, estimating the total number of fruits involves estimating from the counts of several sampled plants because it is impractical to count every single fruit in such vast spaces. Therefore, methods to automatically count the number of fruits using computer vision technologies have been proposed. Parico and Ahamed [3] counted pear fruits from only RGB images using the YOLOv4 [12] model for object detection and DeepSORT [16] (an object tracking model) for fruit counting. Their system is capable of counting pears in real-time at 24 frames per second (fps). Besides, Ge et al. [4] tracked and counted tomatoes at different growth stages using the YOLO-Deepsort network. Their approach is distinctive in that they carefully compared multiple models, including the use of heatmaps. Gao et al. [5] counted apple fruits by combining a YOLOv4-tiny-based object detection network and a CSR-DCF-based tracker. They used YOLOv4-tiny first to detect fruits and tree trunks in each frame and then used the CSR-DCF tracker to calculate displacement between frames. Finally, the current position of the tracked fruits in the present frame was predicted based on the displacement between frames, and the detected fruits in the current frame are associated with the tracked fruits by minimizing the Euclidean distance. One of the contributions of their study is the tracking of tree trunks instead of directly tracking small, difficult-to-track fruits, thereby enabling fruit tracking across multiple frames. These studies propose a method for counting the number of fruits from videos taken in the field, while they do not discuss methods for counting the total number of fruits across the entire field or greenhouse. When filming the entire greenhouse, the same fruit may appear in multiple video frames, leading to a problem of double counting, which prevents an accurate count.

We aim to develop a method not just for a section but for accurately counting the number of fruits throughout the entire greenhouse. Our method allows for counting the number of fruits per ridge, thereby enabling the count of the total number of fruits throughout the greenhouse. Generally, as illustrated in Fig. 2 and Fig. 5, agricultural greenhouses consist of multiple ridges where crops are planted. By counting the number of fruits on each ridge, it is possible to determine the total number of fruits in the entire greenhouse.

More specifically, this paper focuses on counting eggplants as its subject. An RGB camera is mounted on a cart that travels along the aisles between the ridges, capturing images of these ridges. Subsequently, object detection and tracking algorithms



Fig. 2. Greenhouse used for the experiment

are applied to each frame of the captured video to count the eggplants. Finally, by utilizing the principle of motion parallax, eggplants that are not to be counted and appear in the video are removed, enabling the counting of eggplants on a per-ridge basis. Fig. 1 illustrates the overview of our method.

The novelty of our method lies in its ability to separate fruits on the foreground ridges from video footage of multiple ridges, using the principle of motion parallax.

II. OBJECT DETECTION AND TRACKING

Object detection and object tracking methods are essential technologies to accurately count fruits. Object detection identifies objects within a single image, while tracking establishes the continuity of these objects across multiple frames. By integrating both detection and tracking, it is possible to accurately count objects over a sequence of images.

A. Object Detection

Object detection is a task that involves detecting whether objects belonging to predefined categories (such as humans, cars, bicycles, dogs, cats, etc.) are present in an image, and if they are, inferring the position and extent of each object [6]. The position and extent of objects are defined using the smallest rectangle that surrounds the object, called a bounding box. Fig. 3 (a) shows an example of the output from the object detection model, where each eggplant fruit is enclosed in a bounding box.

Since 2012, with the development of deep learning, deep learning-based object detection methods have become mainstream. One notable approach using deep learning is Faster R-CNN [7], which is composed of modules known as Region Proposal Networks (RPN) and Fast R-CNN [8]. The RPN takes any size of image as input and outputs candidate object regions along with an objectness score that indicates the likelihood of an object's presence. These candidate regions are then passed to Fast R-CNN, where the object class and bounding box estimates are determined. Techniques like Faster R-CNN that propose multiple candidate object regions and perform

object class classification on each candidate are referred to as two-stage detection systems.

In contrast to two-stage object detection methods, You Only Look Once (YOLO) [9]–[12] is known as a one-stage method. YOLO first divides the input image into a square grid. Each grid predicts two bounding boxes and a confidence score for each bounding box. The confidence score reflects whether the bounding box contains an object and how accurate the bounding box is. Additionally, each grid predicts the conditional probability that it contains an object of a specific class, given the grid contains any object. Finally, the bounding box's classification is determined by multiplying the conditional probability with the confidence score. YOLO's inference speed is 45 fps, which is nine times faster than Faster R-CNN. However, a limitation of YOLO is that it predicts only two bounding boxes per grid and each grid can represent only one class, making it less effective at recognizing smaller objects.

In our method, we have chosen to use YOLOv5 as our object detection technique, as it is one of the most commonly used methods and offers a balanced trade-off between high detection accuracy and fast inference speeds.

B. Multi-Object Tracking

Multiple object tracking (MOT) is the task of locating multiple objects, identifying each one, and outputting the trajectory of each object in an input video. Examples of objects that can be tracked include pedestrians on city streets. In this paper, we apply MOT techniques to eggplants. Fig. 3 (b) illustrates the application of the object tracking algorithm to eggplant fruits.

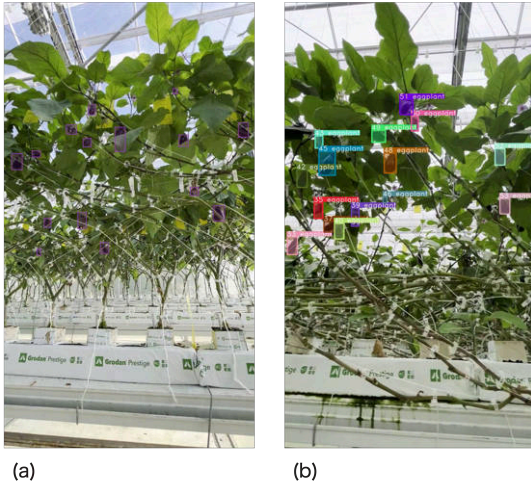


Fig. 3. Object detection example (a). An example of object tracking (b). Each eggplant fruit is assigned a bounding box with a unique ID and a color.

Simple Online and Realtime Tracking (SORT) [13] is an algorithm that accomplishes MOT by linking objects detected as bounding boxes across frames. SORT associates the objects detected in the current frame with the objects being tracked according to the following procedure:

- 1) Predict the position of each tracked object in the current frame.

- 2) Calculate a matrix representing the assignment cost from the detected objects in the current frame and the predicted positions of the tracked objects.
- 3) Solve the assignment problem using the Hungarian algorithm [14].

Here, the prediction of an object's position is performed using a Kalman filter [15]. If an object detected in the current frame is associated with a tracked object, the velocity of the tracked object is estimated using the detected bounding box. Frameworks such as SORT, which perform object tracking using the results of object detection (bounding boxes), are referred to as tracking-by-detection.

DeepSORT [16] is an improved version of SORT that utilizes appearance information of objects. In DeepSORT, detected bounding boxes are first fed into a pre-trained CNN, from which features reflecting the appearance of the object, known as appearance descriptors, are extracted. Subsequently, when associating detected objects with those being tracked, the cosine similarity between appearance descriptors is calculated to reduce the assignment cost for objects with similar appearances.

ByteTrack [17] is an object tracking method that, unlike existing methods which use only bounding boxes with high confidence scores for tracking, utilizes almost all bounding boxes regardless of their confidence scores. In ByteTrack, tracked objects are first associated with bounding boxes that have high confidence scores. If this association fails, the tracked objects are then assigned to bounding boxes with lower confidence scores. ByteTrack's use of bounding boxes with low confidence scores makes it a robust model against occlusions caused by obstacles.

While methods like DeepSORT use appearance information for MOT, distinguishing eggplants based solely on appearance can be challenging. Therefore, our method employs SORT, a relatively simple algorithm that relies solely on the position of objects.

III. OUR METHOD

A. Requirements for our method

The requirements for our method are as follows:

- The method must be applicable in a typical agricultural greenhouse composed of multiple rows of ridges.
- It must be able to count the number of fruits in the entire greenhouse, not just a part of it.
- The method should be applicable specifically to eggplants.
- It must be able to count the number of fruits on a daily basis.
- The method should be executable with commonly available GPU resources, such as those provided by Google Colaboratory [18].

Considering the growth speed of the crops, it is practical to count the number of fruits on a daily basis. Therefore, our method does not require real-time processing, and instead of edge devices, it can utilize cloud environments for computational resources.

B. Details of our method

In our method, by counting the number of fruits on each ridge, we calculate the total number of fruits in the entire greenhouse. The entire ridge is captured using an RGB camera mounted on a trolley (Fig. 4 and Fig. 5). In typical agricultural greenhouses, trolleys designed to travel at a constant speed for applications such as pesticide spraying are available. It is considered that by mounting a camera on the existing trolley, it is possible to capture video and count fruits while other tasks are being performed with the trolley.



Fig. 4. The trolley used for capture (left). A smartphone was used as the RGB camera, which was mounted on the trolley through a stabilizer to reduce image blur caused by the trolley's vibrations due to the motor-driven operation (right).

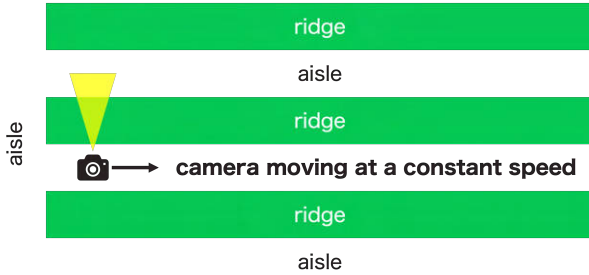


Fig. 5. Method for capturing the entire ridge. When capturing a different ridge, it is necessary to move the trolley to a different rail.

We add a post-processing step to the combination of an existing object detection model and object tracking model, which removes fruits from the ridges that are not targeted for counting from the detection results. The processing pipeline of our method is as follows, in chronological order:

- 1) Video footage of the ridges is captured by an RGB camera equipped on a trolley that travels at a constant speed along the pathways between the ridges.
- 2) The video footage is input into an object detection model frame by frame, obtaining the bounding box of the fruits in each frame.
- 3) The bounding boxes from each frame are input into an object tracking model, which establishes the correspondence of the fruits across the frames.

- 4) The average speed of each fruit in the captured video footage is calculated. Fruits whose average speed is below a certain threshold are determined to be from ridges not targeted for measurement and are removed from the detection results.

To count only the fruits on the ridges in the foreground, we utilize the principle of motion parallax, where closer objects appear to move faster and distant objects move slower. Since the trolley equipped with the camera moves at a constant speed, fruits whose average velocity is below a certain threshold can be removed from the detection results, ensuring that only the fruits on the targeted foreground ridges are counted (Fig. 6).

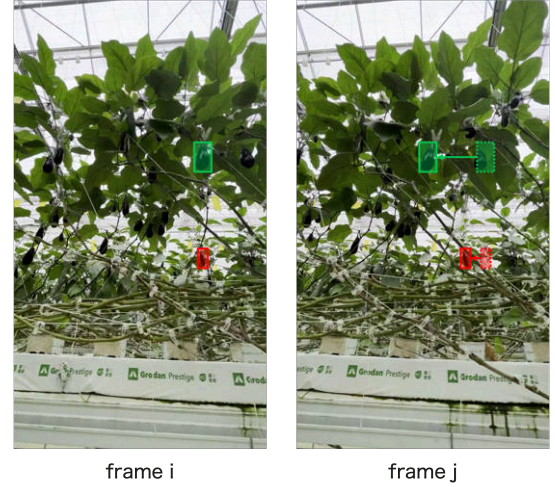


Fig. 6. The difference in displacement between the front fruit and the back fruit. When progressing from frame i to frame j , the fruit enclosed in green rectangles shows greater displacement due to being closer to the camera, whereas the fruit enclosed in red rectangles shows less displacement due to being farther from the camera.

To calculate the average speed of a particular eggplant fruit, we define the frame in which the fruit is first detected as frame i and the next detection frame as frame j . The center coordinates of the fruit's bounding box in frames i and j are (x_i, y_i) and (x_j, y_j) , respectively. The displacement of the fruit between frames i and j is given by:

$$\sqrt{(x_j - x_i)^2 + (y_j - y_i)^2} \quad (1)$$

and the speed is expressed as:

$$\frac{\sqrt{(x_j - x_i)^2 + (y_j - y_i)^2}}{j - i} \quad (2)$$

By calculating the speed for all frames where the fruit is detected and averaging these, the average speed of the fruit is obtained. After calculating the average speed of all fruits, those with an average speed below a threshold are identified as fruits of the background ridge and removed from the detection results. The threshold is a hyperparameter in our method and must be adjusted before using the model.

C. Implementation

We used an iPhone 13 as the RGB camera. The resolution of the captured video was 1080p, and the frame rate was 30 fps. The iPhone 13 was mounted on a trolley, which was driven at a constant speed of approximately 0.4 m/s to capture video footage of the entire ridge.

Each frame of the captured video was processed through the object detection model to obtain the bounding boxes of the eggplants in each frame. We used the YOLOv5 model, which was trained on a dataset initially comprising 180 images of eggplants collected from the Internet and an actual greenhouse. To prepare the model for varying sunlight conditions in the greenhouse, brightness augmentation was applied, doubling the dataset to 360 images. Augmentation refers to the creation of new images for training by transforming existing images, and this time only the brightness was altered. The accuracy of the object detection task was verified using 10 test data samples, where YOLOv5 achieved a mean average precision (mAP) @50 of 0.75. The test data were created by the authors annotating images captured in an actual greenhouse.

To establish the correspondence of detected eggplants across frames, the bounding boxes from each frame were input into an object tracking model. We employed the SORT object tracking algorithm.

Finally, fruits with an average speed below a threshold were removed from the detection results. The threshold was set by using a portion of the created ground truth data for hyperparameter tuning, ensuring that the number of fruits in the ground truth closely matched the number of fruits in the model's output. The actual threshold value was set at 21, which means that fruits moving less than 21 pixels on average between frames were considered as background and removed.

The above process was conducted by uploading videos captured on an iPhone 13 to Google Colaboratory and executing a Python program on Google Colaboratory. The GPU used was an NVIDIA Tesla T4 [19].

IV. EVALUATION

A. Data Collection

The evaluation data were collected from an actual greenhouse in Kanagawa Prefecture, Japan. Data collection was carried out on two separate occasions, with video footage of the same ridge captured each time. Both datasets were recorded during daylight hours, specifically on March 16 and March 23, 2023. From the data collected on both occasions, ten-second sequences of consecutive frames were randomly extracted, and annotations serving as ground truth labels were added to each frame. In total, 596 frames were annotated for evaluation. We labeled those frames using an annotation assistance software called FastLabel [20]. One set of the data was used to adjust the model's hyperparameters, and the other was used for calculating evaluation metrics. In addition to the video data, we manually counted the actual number of fruits on the captured ridge to compare with the model's estimates. The counting was conducted visually from the aisles, moving leaves aside and using a tally counter.

B. Object tracking metrics

Since the count of each individual eggplant fruit involves assigning a unique ID, fruit counting is inherently a task of object tracking. We used object tracking metrics such as IDP (ID precision), IDR (ID recall), and IDF1 (ID F1 score). These identity-based metrics [21] place more emphasis on tracking rather than detection. Tracking accuracy is considered more critical for fruit counting, hence we employed these metrics. Another well-known object tracking metric, MOTA [22], is more influenced by detection rather than tracking, and was therefore deemed inappropriate for this evaluation and not used. The evaluation results of the existing methods and our method, based on identity-based metrics, are shown in Table I.

TABLE I
EVALUATION RESULTS OF EXISTING AND OUR METHODS USING
EVALUATION METRICS FOR OBJECT TRACKING

Method	IDP	IDR	IDF1
SORT	0.462	0.756	0.574
ByteTrack	0.410	0.553	0.471
Ours	0.683	0.741	0.711

Our method builds on the existing SORT algorithm, enhanced by a post-processing step to remove non-target fruits. While this has led to an improvement in precision compared to SORT, the recall has deteriorated. Despite this, the overall F1 score has improved.

C. The number of fruits in the ground truth labels

The number of eggplant fruits appearing in the 10-second frames with annotations was 88. The outputs of the existing method and our method for this are shown in Table II.

TABLE II
COMPARISON WITH THE GROUND TRUTH LABELS

Method	Prediction	Error
SORT	153	0.739
ByteTrack	138	0.568
Ours	85	-0.034

These results show that the existing method predicted more fruits compared to the ground truth, as it also counts the fruits on the ridges in the background of the video.

D. Comparison between the number of fruits visible in the video and the actual number of fruits

On the same day that the evaluation data used in Table II was collected, we confirmed 346 fruits in the targeted ridge. The evaluation data consists of frames captured over a 10-second period, but we determined that it takes 36 seconds to capture the entire ridge with the trolley. Therefore, by multiplying the number of fruits visible in the 10-second video, which is 88, by 3.6, it is estimated that about 317 fruits are visible in the video. This is approximately -8% of the actual number of fruits, which is 346.

V. DISCUSSION

According to Table I, our method outperforms SORT in terms of IDP and IDF1 scores, while it falls short in IDR. This indicates that in the post-processing step of our method, while retaining the true positive detections, it successfully reduces the false positive results. It can be inferred that our method effectively removes only the fruits belonging to the ridges in the background from the detection results.

Despite the actual number of eggplants being 346, the estimated count in the video is slightly lower at 317. This discrepancy can be attributed to fruits being hidden behind leaves and not appearing in the video. In our method, we have mounted an RGB camera on the trolley with a fixed angle. Consequently, it is likely that some fruits will not appear in the video due to blind spots.

According to our results, the SORT object tracking method showed superior results over ByteTrack in all evaluation metrics: IDP, IDR, and IDF1. ByteTrack, proposed as an improvement to SORT, is generally considered to be more accurate. It is speculated that in the use case of this research, where the tracked objects, eggplant fruits, remain stationary, the consistent spacing between the fruits across frames may advantageously influence the Hungarian algorithm employed by SORT.

Our method is considered to be generally applicable for counting stationary objects in agricultural greenhouses. The method utilizes bounding boxes, outputs from an object detection model, to count objects. By simply replacing the object detection model, this method can also be applied to other crops, such as tomatoes, or even to counting flowers instead of fruits.

VI. LIMITATIONS

A limitation of this paper is the small size of the evaluation data. Although evaluations have been conducted using approximately 600 images according to object tracking metrics, and the results of these metrics are somewhat reliable, the data was collected over only two days. This means that the robustness of the model against changes in lighting conditions, such as on cloudy days, has not been adequately tested.

Another limitation mentioned in this paper is the discrepancy between the number of fruits visible in the video and the actual number of fruits present in the greenhouse. In the experiments conducted in this paper, the number of fruits visible in the video was about 8% less than the actual number, often due to being obscured by leaves. To use our method for estimating the actual number of fruits, it is necessary to adjust the model's output values (the count of fruits in the video) by applying a constant factor. This constant for adjustment needs to be estimated through further repeated experiments.

VII. CONCLUSIONS

In this paper, we propose a method to accurately count the number of eggplant fruits throughout the entire greenhouse using a commonly equipped trolley and an inexpensive RGB

camera, by utilizing object detection, object tracking algorithms, and the principles of motion parallax. The results of our evaluation using data collected in an actual greenhouse suggested that our method was superior than existing methods for object tracking and comparisons with the actual number of fruits.

REFERENCES

- [1] International Society of Precision Agriculture. "Precision Ag Definition." International Society of Precision Agriculture. <https://www.ispag.org/about/definition> (accessed May 30, 2024).
- [2] The University of Vermont. "Balancing Vegetative and Reproductive Growth in Greenhouse Tomatoes." The University of Vermont. <https://www.uvm.edu/vtvegandberry/factsheets/VegetativeReproductiveGHTomato.pdf> (accessed Aug. 7, 2024).
- [3] A. I. B. Parico and T. Ahamed, "Real Time Pear Fruit Detection and Counting Using YOLOv4 Models and Deep SORT," *Sensors*, vol. 21, no. 14, p. 4803, July 2021.
- [4] Y. Ge et al., "Tracking and Counting of Tomato at Different Growth Period Using an Improving YOLO-Deepsort Network for Inspection Robot," *Machines*, vol. 10, no. 6, p. 489, June 2022.
- [5] F. Gao et al., "A novel apple fruit detection and counting methodology based on deep learning and trunk tracking in modern orchard," *Computers and Electronics in Agriculture*, vol. 197, no. 107000, June 2022.
- [6] L. Liu et al., "Deep Learning for Generic Object Detection: A Survey," *International Journal of Computer Vision*, vol. 128, pp. 261–318, Sept. 2020.
- [7] S. Ren, K. He, R. Girshick, and J. Sun, "Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks," presented at the Conf. on Neural Information Processing Systems, Cambridge, MA, USA, Dec. 2015, pp. 91–99.
- [8] R. Girshick, "Fast R-CNN," presented at the International Conf. on Computer Vision, Santiago, Chile, Dec. 2015, pp. 1440–1448.
- [9] J. Redmon, S. Divvala, R. B. Girshick, and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," presented at the Conf. on Computer Vision and Pattern Recognition, Las Vegas, USA, June 2016, pp. 779–788.
- [10] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," presented at the Conf. on Computer Vision and Pattern Recognition, Honolulu, USA, July 2017, pp. 6517–6525.
- [11] J. Redmon and A. Farhadi, "YOLOv3: An Incremental Improvement," 2018, arXiv:1804.02767.
- [12] A. Bochkovskiy, C. Wang, and H. M. Liao, "YOLOv4: Optimal Speed and Accuracy of Object Detection," 2020, arXiv:2004.10934.
- [13] A. Bewley, Z. Ge, L. Ott, F. Ramos, and B. Upcroft, "Simple online and realtime tracking," presented at the IEEE Int. Conf. on Image Processing, Phoenix, AZ, USA, Sept. 2016, pp. 3464–3468.
- [14] H. W. Kuhn, "The Hungarian method for the assignment problem," *Naval Research Logistics Quarterly*, vol. 2, no. 1-2, pp. 83–97, Mar. 1955.
- [15] R. E. Kalman, "A New Approach to Linear Filtering and Prediction Problems," *Journal of Basic Engineering*, vol. 82, no. 1, p. 35, Mar. 1960.
- [16] N. Wojke, A. Bewley, and D. Paulus, "Simple online and realtime tracking with a deep association metric," presented at the IEEE Int. Conf. on Image Processing, Beijing, China, Sept. 2017, pp. 3645–3649.
- [17] Y. Zhang et al., "ByteTrack: Multi-object Tracking by Associating Every Detection Box," presented at the European Conf. on Computer Vision, Tel Aviv, Israel, Oct. 2022, pp. 1–21.
- [18] Google. "Welcome To Colab." Google. <https://colab.research.google.com/> (accessed Aug. 15, 2024).
- [19] Nvidia. "NVIDIA T4." Nvidia. <https://www.nvidia.com/en-us/data-center/tesla-t4/> (accessed Aug. 15, 2024).
- [20] FastLabel. "FastLabel." FastLabel. <https://fastlabel.ai/en> (accessed Aug. 15, 2024).
- [21] E. Ristani, F. Solera, R. S. Zou, R. Cucchiara, and C. Tomasi, "Performance Measures and a Data Set for Multi-Target, Multi-Camera Tracking," presented at the European Conf. on Computer Vision, Amsterdam, The Netherlands, Oct. 2016, pp. 17–35.
- [22] K. Bernardin and R. Stiefelhagen, "Evaluating Multiple Object Tracking Performance: The CLEAR MOT Metrics," *EURASIP Journal on Image and Video Processing*, vol. 2008, no. 246309, May 2008.